

TÀI LIỆU KIẾN TRÚC PHẦN MỀM (SOFTWARE ARCHITECTURE DOCUMENT)

Hệ thống Quản lý và Số hóa Tài liệu Thư viện HCMUS (HCMUS-LDMS)

THÔNG TIN TÀI LIỆU (DOCUMENT CONTROL)

Trường thông tin (Field)	Nội dung đặc tả (Description)
Mã tài liệu (Document ID)	HCMUS-LDMS-SAD
Tên tài liệu (Document Title)	Tài liệu Kiến trúc Phần mềm (Software Architecture Document)
Dự án (Project Name)	HCMUS-LDMS
Đơn vị soạn thảo (Author/Organization)	Thư viện & Phòng Công nghệ Thông tin - HCMUS
Người xem xét (Reviewer)	Trưởng phòng CNTT & Giám đốc Thư viện
Người phê duyệt (Approver)	Ban Giám hiệu Trường ĐH Khoa học Tự nhiên
Cấp độ bảo mật (Security Class)	Internal (Nội bộ trường)
Trạng thái tài liệu (Status)	Approved (Đã duyệt)

LỊCH SỬ PHIÊN BẢN (REVISION HISTORY)

Phiên bản (Version)	Ngày phát hành (Date)	Mô tả thay đổi (Description of Change)	Người thực hiện (Author)
1.0	07/07/2026	Khởi tạo tài liệu kiến trúc ban đầu.	Mạch Quốc Tấn
2.0	14/07/2026	Cập nhật cấu trúc phân lớp logic, tích hợp các sơ đồ C4 PlantUML (Context, Container, Deployment), sơ đồ tuần tự, sơ đồ Use Case và phân tích phản biện thực thi.	Mạch Quốc Tấn

Mục lục

- 1. Giới thiệu
 - 1.1 Mục đích
 - 1.2 Phạm vi tài liệu
 - 1.3 Định nghĩa và thuật ngữ viết tắt

- 2. Mục tiêu và Ràng buộc Kiến trúc
 - 2.1 Mục tiêu kiến trúc
 - 2.2 Ràng buộc kiến trúc
- 3. Mô hình Use-Case hệ thống
- 4. Góc nhìn logic (Logical View - C4 Model)
 - 4.1 Phong cách kiến trúc hệ thống
 - 4.2 Ngăn xếp công nghệ chi tiết (Technology Stack)
 - 4.3 Sơ đồ bối cảnh hệ thống (C4 Context Diagram)
 - 4.4 Giải thích các thành phần Kiến trúc logic chi tiết
 - 4.5 Sơ đồ kiến trúc Container (C4 Container Diagram)
 - 4.6 Sơ đồ tuần tự xử lý số hóa và đọc sách (Sequence Diagram)
 - 4.7 Chi tiết các luồng nghiệp vụ cốt lõi
- 5. Giải pháp bảo mật, Sao lưu và Khôi phục thảm họa
 - 5.1 Chiến lược bảo mật dữ liệu (Security Layers)
 - 5.2 Chiến lược sao lưu dữ liệu (Backup Strategy)
 - 5.3 Chỉ số khôi phục thảm họa (Disaster Recovery Targets)
- 6. Góc nhìn Triển khai (Deployment View)
- 7. Góc nhìn Thực thi (Implementation View)
 - 7.1 Cấu trúc thư mục Frontend React
 - 7.2 Cấu trúc thư mục Backend FastAPI
 - 7.3 Lý do lựa chọn cấu trúc thư mục & Phân tích phản biện
- 8. Quản lý cấu hình phần mềm và Chiến lược Git (SCM)

1. Giới thiệu

Tài liệu này trình bày tổng quan cấu trúc thiết kế kiến trúc của hệ thống **HCMUS-LDMS**. Báo cáo làm rõ sự phân chia các tầng trách nhiệm, định hướng công nghệ lựa chọn và cách thức các module tương tác để hiện thực hóa luồng số hóa tự động hóa khép kín.

1.1 Mục đích

Định hình khung kỹ thuật làm cơ sở phát triển đồng bộ cho nhóm lập trình viên Phòng CNTT, đồng thời chứng minh cho Ban Giám hiệu trường về tính khả thi, bảo mật thông tin và khả năng vận hành trơn tru của hệ thống trong điều kiện hạ tầng nội bộ.

1.2 Phạm vi tài liệu

Tài liệu bao phủ toàn bộ kiến trúc logic, sơ đồ triển khai phần cứng ảo hóa, cấu trúc tổ chức mã nguồn và các quyết định kỹ thuật liên quan đến hệ thống HCMUS-LDMS.

1.3 Định nghĩa và thuật ngữ viết tắt

Thuật ngữ	Định nghĩa
LDMS	Library Document Management & Digitization System (Hệ thống quản lý số hóa thư viện).
SSO	Single Sign-On (Xác thực một lần qua tài khoản Keycloak nội bộ).
DRM	Digital Rights Management (Cơ chế quản lý bản quyền số chống sao chép).
UCP	Use Case Points (Phương pháp ước lượng dựa trên điểm trường hợp sử dụng).
COCOMO	Constructive Cost Model (Mô hình toán học ước lượng nỗ lực phần mềm).
SPA	Single Page Application (Ứng dụng web trang đơn).

2. Mục tiêu và Ràng buộc Kiến trúc

2.1 Mục tiêu kiến trúc

- Hiệu năng cao:** Phản hồi tìm kiếm toàn văn Elasticsearch dưới 3 giây dưới tải trọng 500 người dùng đồng thời.
- Bảo mật bản quyền:** Chặn hoàn toàn tải tệp EPUB gốc, chặn chuột phải sao chép văn bản và vô hiệu hóa in ấn tài liệu nội bộ.
- Xử lý bất đồng bộ:** Tác vụ OCR nặng được đẩy xuống Celery worker xử lý dưới nền để không gây nghẽn luồng request chính của API.

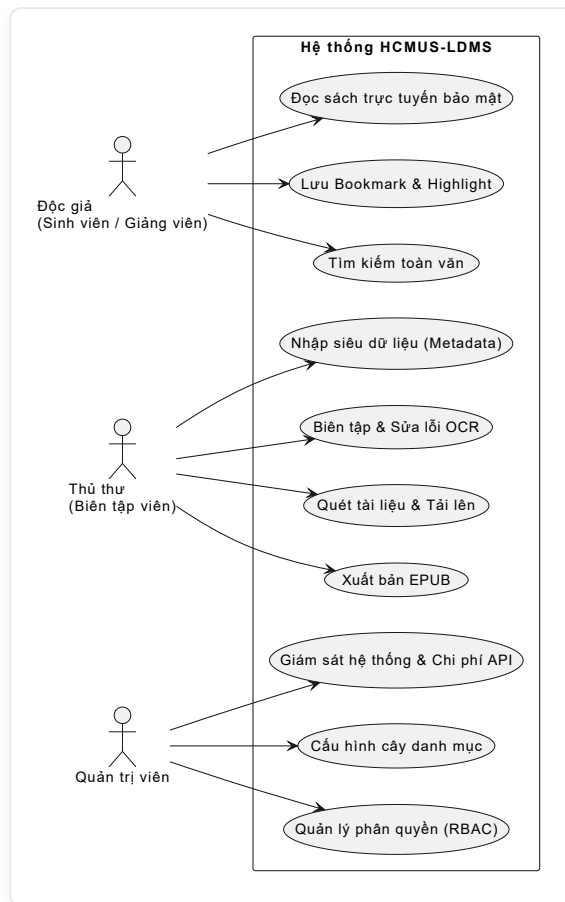
2.2 Ràng buộc kiến trúc

- Hạ tầng On-premise:** Triển khai trên máy chủ VMware vSphere có sẵn của trường HCMUS.
- Ngân sách giới hạn:** Tổng mức chi phí phát triển và mua sắm trang thiết bị ban đầu bắt buộc nằm dưới 100.000.000 VNĐ.

3. Mô hình Use-Case hệ thống

Mối tương tác giữa các tác nhân và các chức năng nghiệp vụ chính của hệ thống được mô tả trực quan qua sơ đồ Use Case PlantUML dưới đây:

Ý nghĩa và lý do cần sơ đồ này: Sơ đồ Use Case đóng vai trò xác định ranh giới chức năng của hệ thống và phân rõ trách nhiệm của các nhóm người dùng (Độc giả, Thủ thư, Quản trị viên). Nó cung cấp cái nhìn tổng quan đầu tiên về các tính năng cần được thiết kế kiến trúc hỗ trợ, đảm bảo mọi ca sử dụng nghiệp vụ đều được ánh xạ đầy đủ sang cấu trúc các module kỹ thuật.

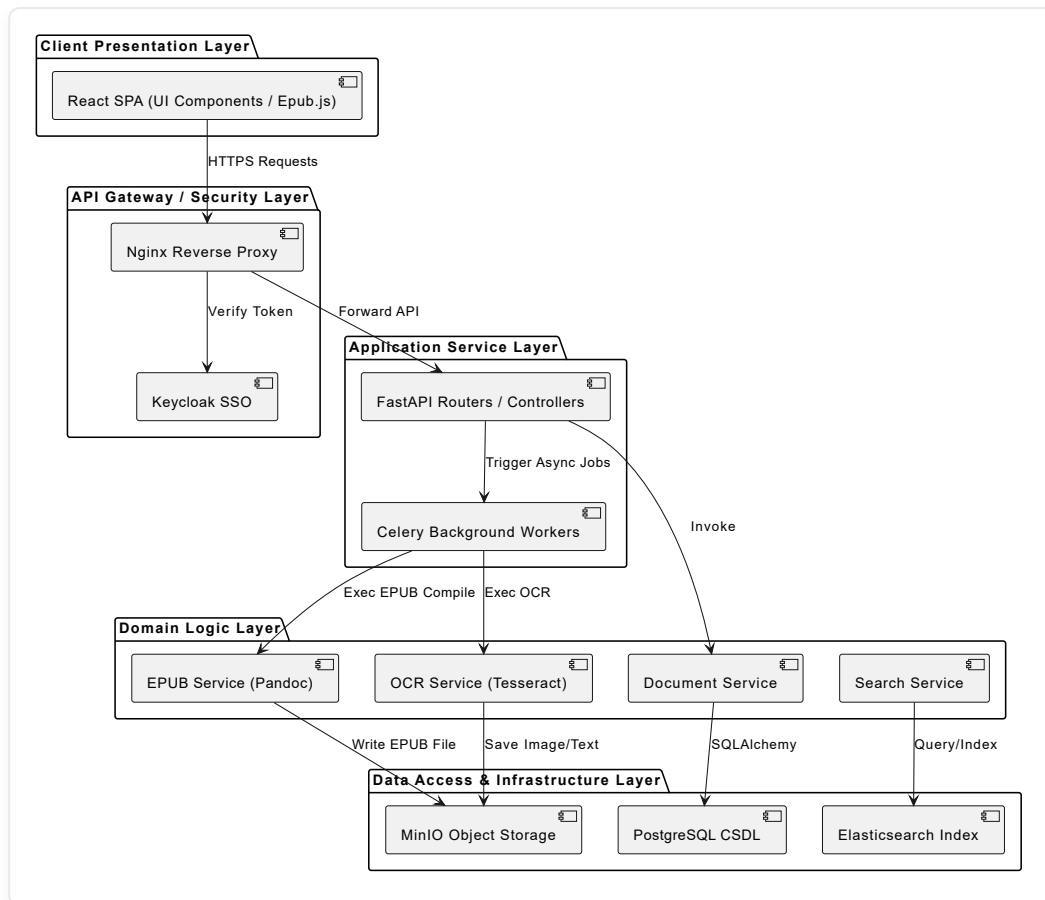


Hệ thống hỗ trợ 3 tác nhân người dùng chính tương tác qua các ca sử dụng cốt lõi: * **Thủ thư / Biên tập viên**: Ca sử dụng *Quét tài liệu, Biên tập lỗi OCR, Nhập Metadata, Xuất bản EPUB*. * **Độc giả (Sinh viên, Giảng viên)**: Ca sử dụng *Đọc trực tuyến bảo mật, Tìm kiếm toàn văn, Lưu Bookmark & Highlight*. * **Quản trị viên (Admin)**: Ca sử dụng *Quản lý phân quyền, Cấu hình danh mục, Giám sát hệ thống*.

4. Góc nhìn logic (Logical View - C4 Model)

Mô tả phân lớp chức năng từ giao diện người dùng, xử lý nghiệp vụ trung gian đến lưu trữ dữ liệu nền tảng được thể hiện qua sơ đồ phân lớp kiến trúc logic dưới đây:

Ý nghĩa và lý do cần sơ đồ phân lớp logic: Sơ đồ phân lớp logic giúp phân rã hệ thống thành các tầng trách nhiệm riêng biệt (Presentation, Gateway/Security, Application Service, Domain Logic, Infrastructure/Data Access). Việc phân tầng này đảm bảo tính độc lập phát triển, giúp kiểm soát tốt các luồng phụ thuộc (dependencies) chiều dọc, cô lập các thay đổi công nghệ cơ sở dữ liệu hoặc framework mà không làm ảnh hưởng đến tầng nghiệp vụ và giao diện người dùng.



4.1 Phong cách kiến trúc hệ thống

Hệ thống HCMUS-LDMS được thiết kế theo mô hình **Modular Monolith** tự xây dựng (custom-built) thay vì sử dụng các phần mềm đóng sẵn có như DSpace hay triển khai kiến trúc Microservices phức tạp.

1. Vì sao không sử dụng DSpace?

DSpace là một kho lưu trữ số tĩnh, tối ưu cho việc lưu trữ file PDF và siêu dữ liệu (metadata) của luận văn, báo cáo nghiên cứu. DSpace hoàn toàn không hỗ trợ các luồng nghiệp vụ tương tác động phức tạp như: chạy OCR tự động nhận dạng ký tự, biên tập sửa lỗi chính tả trực quan so sánh song song với ảnh quét gốc trên giao diện web, và tự động biên dịch sang định dạng EPUB responsive thông qua Pandoc. Việc phát triển một web app custom giúp nhà trường tối ưu hóa hoàn hảo quy trình nghiệp vụ này.

2. Vì sao chọn Modular Monolith thay vì Microservices?

Nhóm phát triển kỹ thuật của Phòng CNTT chỉ gồm 4 kỹ sư làm việc kiêm nhiệm (tương đương 2 lập trình viên full-time). Việc triển khai Microservices sẽ gây quá tải trong khâu quản lý hạ tầng mạng, đồng bộ cơ sở dữ liệu và vận hành CI/CD. Kiến trúc Modular Monolith giúp gói gọn toàn bộ logic nghiệp vụ (OCR, Biên tập, Đóng gói, Tìm kiếm, Đọc sách) vào một khối code duy nhất nhưng được phân chia thành các module độc lập rõ ràng. Điều này giúp đơn giản hóa khâu kiểm thử, triển khai và bảo trì, đồng thời dễ dàng tách nhỏ thành các service độc lập ở giai đoạn sau nếu hệ thống phình to.

3. Xử lý bất đồng bộ các tác vụ nặng (FastAPI & Celery):

Các tác vụ như chạy OCR trên file PDF quét 300 trang hay biên dịch file Markdown sang EPUB bằng Pandoc là các tác vụ rất nặng, chiếm dụng nhiều CPU và thời gian xử lý. Nếu chạy trực tiếp trên luồng API chính sẽ gây nghẽn (block) toàn bộ yêu cầu khác của người dùng. Hệ thống tích hợp hàng đợi Celery kết hợp Redis để xử lý bất đồng bộ các tác vụ này dưới dạng background tasks. API trả về trạng thái "Đang xử lý" ngay lập tức, và giao diện frontend sẽ gửi yêu cầu truy vấn trạng thái (polling) hoặc nhận thông báo qua WebSocket khi tác vụ hoàn thành.

4. Tách biệt Elasticsearch và MinIO Object Storage:

Đảm bảo tính độc lập và hiệu năng cao cho hệ thống. Toàn bộ file scan gốc nặng nề và file EPUB hoàn chỉnh được lưu giữ trên MinIO (S3-compatible Object Storage), giảm tải dung lượng lưu trữ cho máy chủ database PostgreSQL. Trong khi đó, Elasticsearch chịu trách nhiệm lập chỉ mục toàn văn nội dung sách EPUB, bảo đảm tốc độ phản hồi tìm kiếm từ khóa dưới 3 giây.

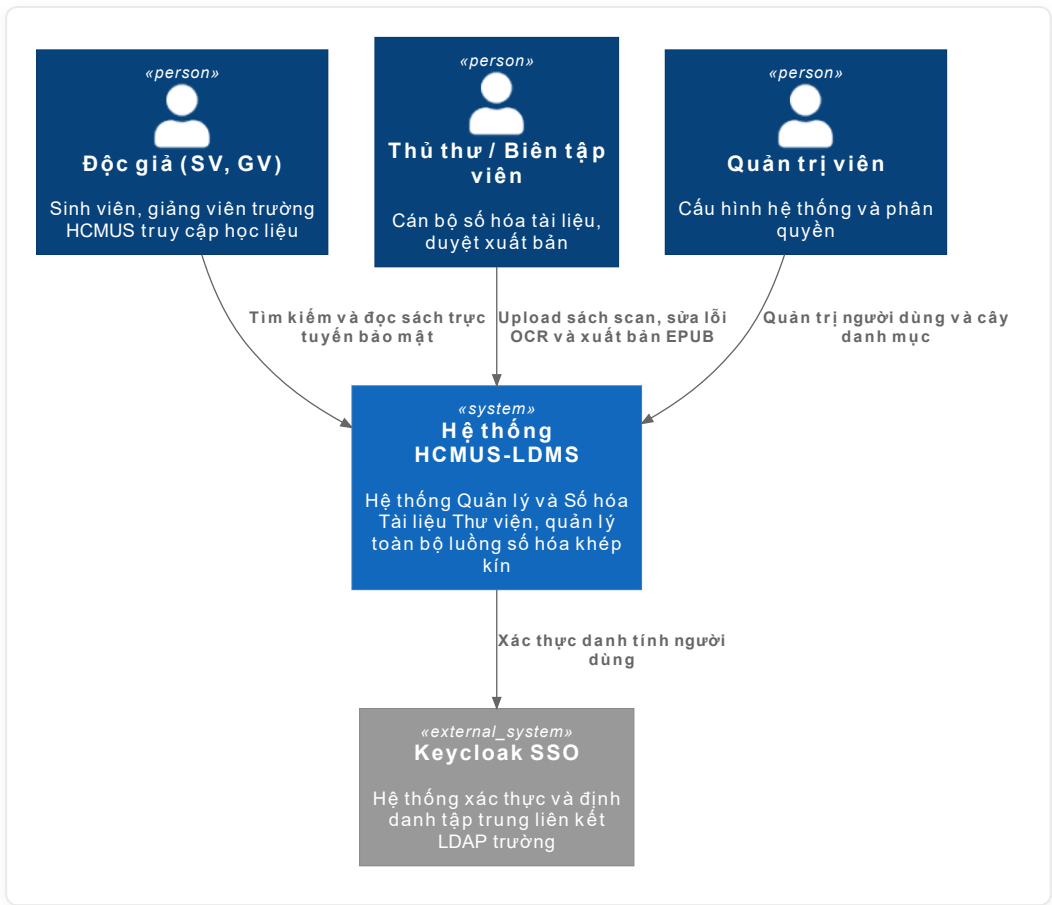
4.2 Ngăn xếp công nghệ chi tiết (Technology Stack)

Thành phần	Lựa chọn công nghệ	Phiên bản	Lý do kỹ thuật
Frontend UI	React 18 & TypeScript	18.3+	Xây dựng giao diện Single Page Application (SPA) phản hồi nhanh, kiểm soát kiểu dữ liệu chặt chẽ tránh lỗi runtime.
Web Reader	Epub.js	0.3+	Thư viện nguồn mở tốt nhất hiện tại giúp hiển thị sách EPUB responsive trực tiếp trên trình duyệt, hỗ trợ bookmark, ghi chú.
Backend API	FastAPI	Python 3.11	Framework API hiệu năng cao, hỗ trợ lập trình bất đồng bộ (async/await), tự động sinh tài liệu Swagger UI.
Web Server	Nginx	1.24+	Reverse Proxy điều phối yêu cầu API, phục vụ static file React và cấu hình mã hóa SSL/TLS 1.3 bảo mật.
Database	PostgreSQL	16	Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, lưu trữ thông tin người dùng, phân quyền RBAC, danh mục và siêu dữ liệu sách.
OCR Engine	Tesseract OCR	5.3+	Engine nhận dạng chữ viết nguồn mở tốt nhất, cấu hình gói tiếng Việt (vie) cài đặt cục bộ trực tiếp trên server backend.
EPUB Compiler	Pandoc & Calibre CLI	3.1+ / 6.0+	Công cụ chuyển đổi định dạng mạnh mẽ, biên dịch Markdown/HTML sang EPUB 3.0 thông qua dòng lệnh.
Tìm kiếm	Elasticsearch	8.11+	Công cụ tìm kiếm phân tán hiệu năng cao, lập chỉ mục và truy vấn toàn văn nhanh chóng.
Lưu trữ tệp	MinIO (On-premise)	RELEASE+	Hệ thống Object Storage tương thích S3, dễ cấu hình và vận hành on-premise trên hạ tầng riêng của trường.
Xác thực	Keycloak	24.x	Máy chủ định danh tập trung (SSO), liên kết trực tiếp với LDAP/Active Directory có sẵn của trường HCMUS.
Triển khai	Docker & Docker Compose	24.0+	Đóng gói toàn bộ các service thành container độc lập giúp dễ dàng phân phối và vận hành trên VMware.
Công cụ Sao lưu	PgBackRest & Restic	2.x	PgBackRest tự động backup gia tăng PostgreSQL hàng ngày. Restic mã hóa và sao lưu MinIO bucket sang máy chủ dự phòng off-site.

4.3. Sơ đồ bối cảnh hệ thống (C4 Context Diagram)

Ý nghĩa và lý do cần sơ đồ này: Sơ đồ bối cảnh (System Context Diagram) là mức cao nhất của mô hình C4 (Mức 1). Nó chỉ rõ vị trí của hệ thống HCMUS-LDMS trong môi trường nghiệp vụ thư viện, làm rõ các tác nhân bên ngoài tương tác trực tiếp (Độc

giả, Thủ thư, Quản trị viên) và các hệ thống bên ngoài liên kết (Keycloak SSO kết nối LDAP trường), giúp người đọc có cái nhìn tổng quan đầu tiên về ranh giới hệ thống.



4.4. Giải thích các thành phần Kiến trúc logic chi tiết

Để làm rõ cấu trúc bên trong các sơ đồ container, dưới đây là chi tiết vai trò, nhiệm vụ và lý do cần thiết của từng thành phần con trong hệ thống:

4.3.1. Phân rã thành phần Frontend Client (React & TypeScript)

- **UI Components (Tầng giao diện):**
 - *Nhiệm vụ:* Chứa các components React có khả năng tái sử dụng (Atomic Design). Nổi bật nhất là màn hình *Split-screen Editor* và *Web Reader (Epub.js)*.
 - *Tại sao cần:* Giúp chia nhỏ giao diện thành các phần độc lập, dễ kiểm thử và phát triển song song. Đảm bảo tính nhất quán của UI/UX trên toàn bộ hệ thống.
- **API Services (Tầng dịch vụ API):**
 - *Nhiệm vụ:* Đóng gói các yêu cầu HTTP (sử dụng thư viện `axios`) gửi đến Backend API Server.
 - *Tại sao cần:* Tách biệt logic giao tiếp mạng ra khỏi UI components, giúp mã nguồn sạch hơn, dễ thay đổi cấu hình Endpoint và tái sử dụng các phương thức gọi dữ liệu.
- **State Management (Tầng quản lý trạng thái - Zustand/Context):**
 - *Nhiệm vụ:* Lưu trữ và quản lý trạng thái dùng chung toàn cục như thông tin người dùng đã đăng nhập (User Profile), phiên xác thực (Auth Token), và thiết lập hiển thị (Sepia/Dark mode).
 - *Tại sao cần:* Tránh hiện tượng "prop drilling" (truyền dữ liệu qua quá nhiều tầng component trung gian), đảm bảo dữ liệu hiển thị đồng bộ trên mọi màn hình.

4.3.2. Phân rã thành phần Backend API Server (FastAPI)

- **Router Layer (Tầng định tuyến):**

- *Nhiệm vụ:* Ánh xạ các HTTP requests (GET, POST, PUT, DELETE) gửi tới các Endpoint (ví dụ: `/api/documents/`) đến đúng Controller xử lý tương ứng.
- *Tại sao cần:* Là cổng vào duy nhất của API, giúp cấu hình định tuyến tập trung và mạch lạc.

- **Middleware Layer (Tầng kiểm soát trung gian):**

- *Nhiệm vụ:* Thực thi các logic kiểm tra chéo (Cross-cutting Concerns) như xác thực JWT Keycloak, phân quyền dựa trên vai trò (RBAC) và kiểm tra kích thước file upload.
- *Tại sao cần:* Chặn đứng các yêu cầu không hợp lệ ngay từ vòng ngoài trước khi chúng chạm vào cơ sở dữ liệu hoặc logic nghiệp vụ chính, bảo vệ tài nguyên hệ thống.

- **Controller Layer (Tầng điều phối):**

- *Nhiệm vụ:* Tiếp nhận request từ Router, thực hiện kiểm tra định dạng dữ liệu (sử dụng Pydantic Schemas), điều hướng sang Service layer xử lý và trả về HTTP response.
- *Tại sao cần:* Đóng vai trò là cầu nối trung gian để chuẩn hóa giao tiếp đầu vào/đầu ra của hệ thống.

- **Service Layer (Tầng nghiệp vụ):**

- *Nhiệm vụ:* Nơi chứa 100% logic nghiệp vụ cốt lõi của dự án (tính toán nỗ lực, gọi engine Tesseract OCR, gọi Pandoc đóng gói sách EPUB, lập chỉ mục Elasticsearch).
- *Tại sao cần:* Đây là lõi nghiệp vụ của toàn dự án, tách biệt hoàn toàn khỏi cơ chế web (HTTP) giúp dễ dàng viết Unit Test và bảo trì lâu dài.

- **Repository Layer (Tầng truy cập dữ liệu):**

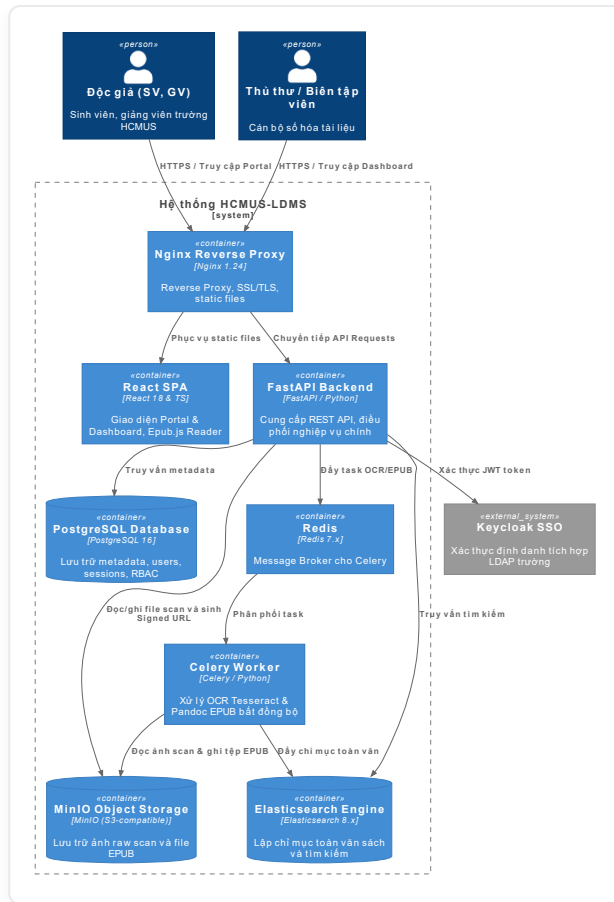
- *Nhiệm vụ:* Sử dụng SQLAlchemy ORM để truy vấn, cập nhật dữ liệu vào PostgreSQL Database.
- *Tại sao cần:* Tránh việc viết các câu lệnh SQL thô phức tạp trong mã nguồn nghiệp vụ, đồng thời trừu tượng hóa các tương tác với CSDL.

- **Model Layer (Tầng thực thể):**

- *Nhiệm vụ:* Định nghĩa cấu trúc các bảng dữ liệu trong PostgreSQL (như `Document`, `User`, `ScanPage`, `Category`, `Tag`).
- *Tại sao cần:* Khớp nối cấu trúc thực thể trong code với lược đồ vật lý của CSDL.

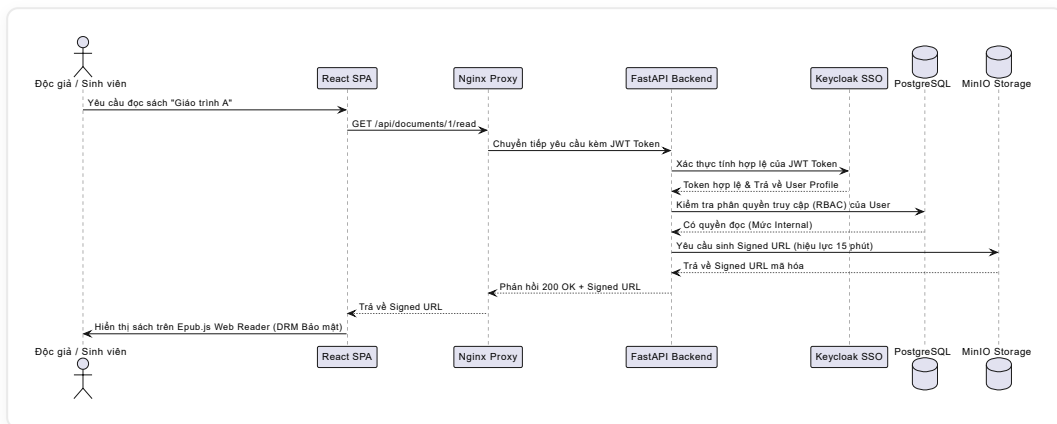
4.3 Sơ đồ kiến trúc Container (C4 Container Diagram)

Ý nghĩa và lý do cần sơ đồ này: Sơ đồ Container C4 cung cấp cái nhìn tổng quan về cấu trúc các thành phần phần mềm (container) độc lập trong hệ thống, chỉ rõ công nghệ được sử dụng (React, FastAPI, PostgreSQL, MinIO, Elasticsearch, Redis, Celery) và cách chúng giao tiếp với nhau. Sơ đồ này giúp các lập trình viên và quản trị viên hệ thống hiểu rõ ranh giới của từng dịch vụ, cách thức luồng dữ liệu đi qua reverse proxy Nginx và phân tách trách nhiệm giữa luồng API đồng bộ và hàng đợi Celery bất đồng bộ.



4.6 Sơ đồ tuần tự xử lý số hóa và đọc sách (Sequence Diagram)

Ý nghĩa và lý do cần sơ đồ này: Sơ đồ tuần tự làm rõ trình tự thời gian và luồng gọi API giữa các thành phần khi thực hiện tác vụ nghiệp vụ quan trọng nhất: Yêu cầu đọc sách bảo mật của sinh viên. Nó chỉ rõ cơ chế kiểm tra phân quyền RBAC dựa trên JWT Token được Keycloak SSO cấp, quá trình đối chiếu quyền trong database PostgreSQL, và việc sinh đường dẫn ký số Signed URL có thời hạn (15 phút) từ MinIO Storage để ngăn chặn hành vi tải lậu trực tiếp.



4.7. Chi tiết các luồng nghiệp vụ cốt lõi

4.7.1. Quy trình Số hóa và Xuất bản (Thủ thư & Biên tập viên)

- Scan sách:** Thủ thư quét sách giấy vật lý thành tệp PDF chất lượng cao (300 DPI, thẳng hàng).
- Tải lên & Khai báo:** Thủ thư đăng nhập hệ thống qua Keycloak SSO, tải file PDF lên và nhập siêu dữ liệu Dublin Core.

3. **OCR nhận dạng chữ:** Backend FastAPI đưa PDF vào hàng đợi Celery, chạy OCR Tesseract nhận dạng tiếng Việt bất đồng bộ để trích xuất văn bản thô.
4. **Biên tập Split-screen:** Biên tập viên sử dụng màn hình chia đôi so sánh ảnh scan trang sách gốc và văn bản OCR, sửa lỗi chính tả, gán Heading và chèn ảnh minh họa.
5. **Phân loại & Phân quyền:** Gán Category hình cây, nhãn Tag và thiết lập quyền truy cập (Public/Internal/Restricted).
6. **Đóng gói & Xuất bản:** Hệ thống gọi Pandoc biên dịch văn bản sang EPUB 3.0, lưu vào MinIO, lưu metadata vào PostgreSQL và index toàn văn sang Elasticsearch.

4.7.2. Quy trình Tìm kiếm và Đọc sách trực tuyến (Sinh viên / Độc giả)

1. **Đăng nhập:** Sinh viên truy cập Web Portal, đăng nhập Keycloak SSO trường.
2. **Tìm kiếm:** Nhập từ khóa tìm kiếm toàn văn Elasticsearch để truy vấn sâu nội dung sách EPUB, kết hợp bộ lọc Category và Tag để lọc kết quả.
3. **Xác thực quyền đọc:** Khi độc giả bấm đọc, backend kiểm tra phân quyền RBAC. Nếu hợp lệ, backend sinh Signed URL MinIO (hiệu lực 15 phút).
4. **Đọc sách responsive:** Trình xem Epub.js hiển thị sách responsive trên mobile/tablet, cho phép chỉnh font, cỡ chữ, màu nền, bookmark, highlight.

5. Giải pháp bảo mật, Sao lưu và Khôi phục thảm họa

5.1 Chiến lược bảo mật dữ liệu (Security Layers)

- **Mã hóa truyền tải:** Cấu hình HTTPS (TLS 1.3) bắt buộc trên Nginx cho mọi luồng truyền tải dữ liệu. Cấu hình tiêu đề bảo mật HSTS (HTTP Strict Transport Security) để ngăn chặn các cuộc tấn công hạ cấp giao thức.
- **Bảo mật truy cập tệp tin (Signed URL):** File EPUB gốc được lưu giữ riêng tư trên MinIO, không công khai URL trực tiếp. Khi độc giả được xác thực quyền đọc sách qua RBAC, Backend FastAPI sẽ sinh một **Signed URL** giới hạn thời gian hiệu lực tối đa là **15 phút**. Trình đọc Epub.js ở frontend dùng URL này để tải file tương ứng của từng chương sách.
- **Vô hiệu hóa tải lậu (Anti-scraping):** Trình xem Epub.js được tích hợp script vô hiệu hóa nhấp chuột phải, chặn phím tắt chụp màn hình (**PrintScreen**), phím tắt sao chép (**Ctrl+C**), chặn in ấn (**Ctrl+P**) và hoàn toàn không hiển thị nút tải file EPUB gốc để bảo vệ quyền tác giả.

5.2 Chiến lược sao lưu dữ liệu (Backup Strategy)

- **PostgreSQL Database:** Sử dụng công cụ **PgBackRest** tự động chạy sao lưu gia tăng (incremental backup) vào lúc 02:00 AM mỗi ngày, và chạy sao lưu toàn phần (full backup) vào tối Chủ nhật hàng tuần. Các bản sao lưu được nén và mã hóa bằng thuật toán AES-256 trước khi lưu trữ vào cụm lưu trữ độc lập.
- **MinIO Object Storage:** Sử dụng công cụ **Restic** tự động đồng bộ hóa và mã hóa toàn bộ dữ liệu PDF gốc và file EPUB thành phẩm hàng đêm sang hệ thống NAS lưu trữ off-site của trường.
- **Chính sách lưu giữ (Retention Policy):** Bản sao lưu ngày giữ trong 30 ngày; bản sao lưu tuần giữ trong 12 tuần; bản sao lưu tháng giữ trong 12 tháng.

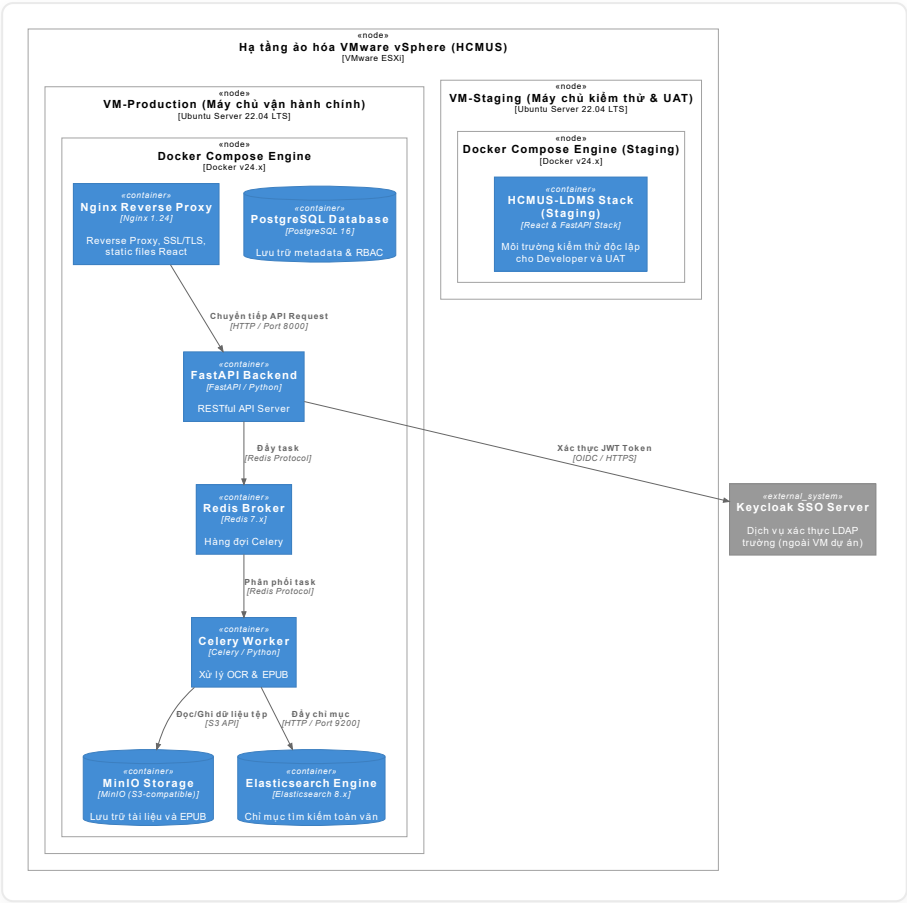
5.3 Chỉ số khôi phục thảm họa (Disaster Recovery Targets)

- **RPO (Recovery Point Objective):** Tối đa **24 giờ**. Đảm bảo trong tình huống xấu nhất (cháy nổ máy chủ vật lý), thư viện không bị mất quá 1 ngày làm việc số hóa của các biên tập viên.

- **RTO (Recovery Time Objective):** Tối đa **4 giờ**. Thời gian khôi phục toàn bộ các container và nạp dữ liệu backup từ NAS sang máy chủ dự phòng để hệ thống hoạt động bình thường trở lại.

6. Góc nhìn Triển khai (Deployment View)

Mô tả phân bố vật lý và cấu trúc mạng ảo hóa của các container trên hệ thống máy chủ trường ĐH KHTN được thể hiện qua sơ đồ triển khai C4 PlantUML:



7. Góc nhìn Thực thi (Implementation View)

7.1 Cấu trúc thư mục Frontend React

```
/frontend
├── public/           # Các tệp tĩnh phục vụ trực tiếp
├── src/
│   ├── assets/      # Font chữ, hình ảnh, CSS dùng chung
│   ├── components/  # Component UI nhỏ gọn, tái sử dụng
│   ├── pages/       # Giao diện chính (Dashboard, Editor, Reader)
│   ├── services/    # Kết nối REST API (auth, document, search)
│   ├── context/     # Lưu trữ global state (phiên đăng nhập, UI mode)
│   ├── utils/       # Các hàm hỗ trợ
│   ├── App.tsx      # Điểm khởi chạy React
│   └── main.tsx     # Điểm gắn kết DOM
├── package.json     # Thư viện phụ thuộc
└── tsconfig.json    # Cấu hình TypeScript
```

7.2 Cấu trúc thư mục Backend FastAPI

```
/backend
├── app/
│   ├── api/         # Định nghĩa Router và Controller
│   │   ├── auth.py  # Xác thực JWT với Keycloak
│   │   ├── document.py # Quản lý số hóa tài liệu
│   │   └── search.py # Tìm kiếm Elasticsearch
│   ├── core/        # Cấu hình hệ thống và kết nối DB
│   ├── models/      # Định nghĩa cấu trúc PostgreSQL (SQLAlchemy)
│   ├── schemas/     # Định nghĩa Pydantic validation
│   ├── services/    # Xử lý nghiệp vụ chính
│   │   ├── ocr.py   # Tích hợp Tesseract
│   │   └── epub.py  # Gọi Pandoc đóng gói
│   └── workers/     # Cấu hình hàng đợi Celery / Redis
├── requirements.txt  # Các thư viện Python phụ thuộc
└── docker-compose.yml # Cấu hình chạy cụm container
```

7.3. Lý do lựa chọn cấu trúc thư mục & Phân tích phản biện (Design Rationale & Refutation)

Để bảo vệ thiết kế cấu trúc thư mục thực thi của dự án trước các phương án thay thế, dưới đây là chi tiết phân tích phản biện:

1. Đối với cấu trúc thư mục Frontend React:

2. *Lựa chọn thiết kế:* Cấu trúc phân lớp kỹ thuật (Technical Layering: `components/`, `pages/`, `services/`, `context/`).

3. *Phản biện (Counter-argument):* Một số lập trình viên cho rằng nên áp dụng cấu trúc phân rã theo tính năng nghiệp vụ (Feature-based structure: ví dụ `/features/ocr/`, `/features/reader/` tự chứa các component, service riêng).

4. *Giải trình biện hộ (Refutation):* Phương án chia theo tính năng (Feature-based) chỉ hiệu quả đối với các hệ thống siêu lớn (Enterprise) với hàng chục lập trình viên làm việc độc lập trên các phân hệ khác nhau. Với dự án HCMUS-LDMS giai đoạn MVP có quy mô nhỏ (dưới 10.000 dòng code frontend), cấu trúc phân lớp kỹ thuật (Technical Layering) giúp:

- *Tái sử dụng tối đa components:* Trình biên tập Split-screen và Trình đọc Web Reader Epub.js sử dụng chung rất nhiều components dùng chung (nút bấm, hộp thoại, thanh điều hướng). Việc gom chung vào `components/` giúp tìm kiếm

và tái sử dụng dễ dàng.

- *Giảm thiểu import chéo phức tạp*: Hạn chế lỗi tham chiếu vòng (circular dependency) thường gặp khi chia nhỏ thư mục theo tính năng.
- *Đường cong học tập thấp*: Phù hợp với nhóm phát triển nhỏ 4 người, giúp dễ dàng định vị file cần sửa đổi.

5. Đối với cấu trúc thư mục Backend FastAPI:

6. *Lựa chọn thiết kế*: Kiến trúc phân tầng 3 lớp (3-tier Architecture: `/api`, `/services`, `/models`).

7. *Phản biện (Counter-argument)*: Ý kiến trái chiều cho rằng nên áp dụng kiến trúc lục giác (Hexagonal Architecture / Clean Architecture) để cách ly hoàn toàn business logic khỏi các framework bên ngoài (SQLAlchemy ORM, FastAPI) thông qua các lớp Interfaces và Adapters.

8. *Giải trình biện hộ (Refutation)*: Mặc dù kiến trúc lục giác tăng tính độc lập kiểm thử, nó đòi hỏi viết rất nhiều lớp trung gian (DTOs, Mappers, Interface classes) làm tăng số lượng dòng code boilerplate vô nghĩa lên gấp 2-3 lần. Trong bối cảnh dự án học thuật giới hạn thời gian (20 tuần) và nhân sự bán thời gian:

- *Tối ưu hóa thời gian phát triển*: Kiến trúc 3 lớp kết hợp SQLAlchemy ORM giúp lập trình viên viết code nghiệp vụ và kết nối DB cực nhanh, đẩy nhanh tiến độ bàn giao sản phẩm.
- *Tránh thiết kế quá tải (Over-engineering)*: Hệ thống LDMS sử dụng PostgreSQL cố định, không có nhu cầu thay đổi hệ quản trị cơ sở dữ liệu trong suốt vòng đời dự án. Do đó, việc xây dựng các adapter trung gian chỉ để độc lập DB là không thực tế và lãng phí nỗ lực.

8. Quản lý cấu hình phần mềm và Chiến lược Git (SCM)

8.1. Quy chuẩn định danh các Hạng mục cấu hình (Configuration Items - CIs)

Để đảm bảo khả năng truy vết và kiểm soát phiên bản trong quá trình phát triển, dự án áp dụng quy tắc đặt mã định danh duy nhất cho các tài liệu và sản phẩm bàn giao: `LDMS_[LOAI_CI]_[TRANG_THAI].X.Y`

Trong đó: * `LDMS`: Mã dự án (HCMUS-LDMS). * `LOAI_CI`: Phân loại hạng mục cấu hình: * `PLN`: Kế hoạch dự án (Project Plan). * `SRD`: Yêu cầu hệ thống (System Requirements). * `SDD`: Thiết kế kiến trúc & hệ thống (System Design). * `SRC`: Mã nguồn ứng dụng (Source Code). * `TSP`: Kế hoạch kiểm thử (Test Plan). * `TRANG_THAI`: Ký hiệu trạng thái phát triển: `R` (Release - Chính thức), `A` (Alpha - Sơ khởi), `B` (Beta - Thử nghiệm). * `X.Y`: Phiên bản chính (X) và phụ (Y).

Ví dụ: * Tài liệu kiến trúc này: `LDMS_SDD_R3.0.md` * Mã nguồn Backend FastAPI: `LDMS_SRC_B1.0`

8.2. Chiến lược nhánh Git (Git Branching Strategy)

Dự án áp dụng mô hình **GitFlow** để điều phối mã nguồn của nhóm phát triển 4 người: * `main`: Nhánh chứa mã nguồn ổn định nhất đang chạy trên môi trường Production. Chỉ được phép merge từ nhánh `release/*` thông qua Pull Request (PR) có sự phê duyệt của Tech Lead (PM/SA). Mỗi lần merge lên `main` sẽ tự động đánh tag phiên bản (ví dụ: `v1.0.0`). * `develop`: Nhánh tích hợp chính cho các tính năng mới trong từng Sprint. Tất cả các developer sẽ tích hợp mã nguồn tại đây sau khi code của họ được review. * `feature/*`: Các nhánh phát triển tính năng riêng lẻ (ví dụ: `feature/ocr-integration`, `feature/epub-reader`). Tạo ra từ nhánh `develop` và merge lại vào `develop` sau khi hoàn tất kiểm thử unit test cục bộ. * `release/*`: Nhánh chuẩn bị phát hành phiên bản (ví dụ: `release/v1.0-MVP`). Nhánh này phục vụ giai đoạn UAT và fix lỗi cuối cùng trước khi chuyển giao chính thức sang `main` và `develop`. * `hotfix/*`: Các nhánh sửa lỗi khẩn cấp trực tiếp từ `main` khi có lỗi bảo mật hoặc sập hệ thống trên Production. Sau khi sửa xong sẽ được merge đồng thời vào `main` và `develop`.